Sistemas Paralelos - 2026

Trabajo Práctico Nº 3
Optimización del rendimiento de un hilo

Facultad de Informática

Objetivos de la práctica:

Comprender cómo diferentes transformaciones del código fuente influyen en el desempeño de un programa, analizando su impacto en el uso del procesador y de la jerarquía de memoria. En particular, se aborda la reducción del trabajo computacional, la mejora del acceso a datos, el incremento del paralelismo de nivel de instrucciones (ILP) y la explotación del paralelismo de datos mediante instrucciones vectoriales (SIMD).

Bibliografía:

- Introduction to High Performance Computing for Scientists and Engineers. CRC Press. Hager, Wellein. 2011.
- Temas de jerarquía de memoria:
  - Del libro de Grama: 2.2.1 y 2.2.2.
  - Lecturas alternativas en sección 2.1.2 del libro de Dongarra y 2.7 del libro de Rauber.
- Software Optimization Guide for AMD Family 15h Processors. https://www.amd.com/content/dam/amd/en/documents/archived-tech-docs/software-optimization-guides/47414_15h_sw_opt_guide.pdf
- Intel 64 and IA-32 Architectures Optimization Reference Manual https://cdrdv2-public.intel.com/814198/248966-Optimization-Reference-Manual-V1-049.pdf
- Intel Intrinsics Guide: https://software.intel.com/sites/landingpage/IntrinsicsGuide/

Instrucciones para la resolución de los ejercicios:

a) Para poder observar de forma aislada el efecto de las optimizaciones manuales realizadas en los ejercicios, compilar los programas sin utilizar las optimizaciones automáticas del compilador.

b) No realizar optimizaciones adicionales (por ejemplo, simplificaciones matemáticas) a las indicadas en cada ejercicio.

c) Para medir el tiempo de ejecución de los programas, anteponer el comando *time* al nombre de su programa ejecutable en el script. Observar el tiempo indicado como “real”, que indica el tiempo real de ejecución tal como si se midiera con un cronómetro, dentro del archivo de error.

d) Preferentemente, utilizar el cluster para ejecutar los programas.

Ejercicios

1) Describir los niveles de optimización 0, 2 y 3 de GCC o ICC.

2) Técnica: “Hacer menos trabajo y más liviano”.

a) Solamente aplicando el sentido común, hacer que el siguiente código de programa realice menos trabajo. Dado que el beneficio de esta técnica es trivial, no se evaluará el tiempo de ejecución de este programa (además, por esta razón, el código está incompleto).

```
#include <stdio.h>

int flag = 0;
for (i = 0; i < 1000; i++) {
```

```
   if (complex_func(a[i]) < 55)
      flag = 1;
}
printf("¿Elemento encontrado? %d\n", flag);
```

b) Utilizando operaciones más livianas (menos complejas), y sin modificar la estructura del algoritmo, optimizar el siguiente código de programa. Contrastar el tiempo de ejecución de la versión optimizada frente a la versión original.

```
int main() {
   float i, j, a;
   for (i = 0; i < 1000; i++)
      for (j = 0; j < 1000000; j++)
         a = i + j;
   return 0;
}
```

3) Técnica: "Reducción del almacenamiento para datos".

El siguiente programa representa una grilla con células vivas y muertas del Juego de la Vida. El programa arma una estructura de datos que permite almacenar el estado de cada célula (cero para célula muerta y uno para célula viva).

a) Realizar un diagrama que permita observar punteros y bloques de memoria de la estructura de datos utilizada para representar la grilla. ¿Qué ventajas tiene el uso de esa estructura frente a una declaración de una matriz estándar?

b) Optimizar el programa mediante la técnica de reducción del almacenamiento para datos.

c) Contrastar el programa optimizado frente a la versión original en relación al tiempo de ejecución y memoria utilizada para almacenar la matriz.

```
#include <stdlib.h>
#define X 20000
#define Y 20000

int main() {
   int i, j;
   int **m = malloc(X * sizeof(int*));
   int *x = malloc(X * Y * sizeof(int));
   for (i = 0; i < X; i++)
      m[i] = &x[i*Y];

   for(i = 0; i < X; i++)
      for(j = 0; j < Y; j++)
         m[i][j] = j % 2;
   return 0;
}
```

4) Técnica: “Código en Línea”.

Optimizar el siguiente programa utilizando la técnica de Código en Línea (se requiere eliminar la función y escribir el código en donde se encuentra cada a la función). Contrastar el tiempo de ejecución de la versión optimizada frente a la versión original. ¿En qué casos cree conveniente utilizar funciones y en qué casos no? analizar en función de: funciones grandes y pequeñas, posibilidad de optimizaciones globales, incremento del tamaño del código.

```
int pordos (int x) {
   return x * 2;
}

int main() {
   register unsigned int i, res = 0;
   for (i = 0; i < 500000000; i++)
      res += pordos(i);
   return 0;
}
```

5) Técnica: “Desenrollado de bucles”.

El programa 1 es el original. El programa 2 resulta de aplicar la técnica de desenrollado de bucles. El programa 3, luego de aplicar desenrollado de bucles, aplicó una técnica para reducir dependencias entre instrucciones dentro de cada iteración. Ejecutar los programas y, tras observar los tiempos de ejecución de cada uno, formular hipótesis que expliquen las diferencias de tiempos.

Programa 1

```
#include <stdio.h>
int main() {
   register int i;
   register double a = 0;
   for (i = 0; i < 40000000; i++)
      a+= 0.0000001;

   printf("El valor calculado es: %f\n", a);
   return 0;
}
```

Programa 2

```
#include <stdio.h>
int main() {
   register int i;
   register double a;
   for (i = 0; i < 40000000; i+=4) {
      a+= 0.0000001;
```

```
      a+= 0.0000001;
      a+= 0.0000001;
      a+= 0.0000001;
   }

   printf("El valor calculado es: %f\n", a);
   return 0;
}
```

Programa 3

```
#include <stdio.h>
int main() {
   register int i;
   register double a = 0, a1 = 0, a2 = 0, a3 = 0;
   for (i = 0; i < 40000000; i+=4) {
         a+= 0.0000001;
         a1+= 0.0000001;
         a2+= 0.0000001;
         a3+= 0.0000001;
   }

   a+=a1+a2+a3;
   printf("El valor calculado es: %f\n", a);
   return 0;
}
```

6) Técnica: “Extracción de subexpresiones comunes”.

Optimizar el siguiente programa extrayendo la subexpresión común. Contrastar el tiempo de ejecución de la versión optimizada frente a la versión original.

```
#include <stdio.h>
int main() {
   register int i, j;
   register int a = 0, a1 = 0, a2 = 0, a3 = 0;
   for (i = 0; i < 1000; i++)
      for (j = 0; j < 1000000; j+=4) {
         a += i * i * i + j;
         a1 += i * i * i + j + 1;
         a2 += i * i * i + j + 2;
         a3 += i * i * i + j + 3;
      }
```

```
   a += a1 + a2 + a3;
   printf("El valor calculado es: %d\n", a);
   return 0;
}
```

7) Técnica: “Evitar saltos condicionales”

Optimizar el siguiente programa mediante la evitación de saltos. Contrastar el tiempo de ejecución de la versión optimizada frente a la versión original. Formular hipótesis que expliquen las diferencias observadas en los tiempos de ejecución.

```
#include <stdlib.h>
#define X 1000000

int main() {
   int i, j, **m;
   m = malloc(900 * sizeof(int*));
   for (i = 0; i < 900; i++)
      m[i] = malloc(X * sizeof(int));
   for(i = 0; i < 900; i++)
      for(j = 0; j < X; j++)
         switch (i) {
            case 0 ... 299: m[i][j] = 0; break;
            case 300 ... 599: m[i][j] = 1; break;
            default: m[i][j] = 2; break;
         }
   return 0;
}
```

8) Técnica: “Uso eficiente de la caché: localidad espacial y temporal”.

a) Optimizar el siguiente programa buscando aumentar la localidad espacial y temporal en el acceso a datos de una matriz. Contrastar el tiempo de ejecución de la versión optimizada frente a la versión original.

b) Para el procesador utilizado en el cluster, indicar los tamaños de:
   i) memoria caché de nivel 3.
      `$ cat /sys/devices/system/cpu/cpu0/cache/index2/size`
      o
      `$ lscpu`
   ii) línea de caché de nivel 3.
      `$ cat /sys/devices/system/cpu/cpu0/cache/index2/coherency_line_size`

c) Calcular, de manera analítica, cuántos fallos de caché hay en cada una de las dos versiones del programa (original y optimizada). Para simplificar, consideraremos solamente los fallos de caché de nivel 3, y un tamaño de 128 KiB para ese nivel.

```
#include <stdlib.h>

#define X 50000
```

```
#define Y 10000

int main() {
   int i, j;
   int **m;
   m = malloc(X * sizeof(int*));
   for (i = 0; i < X; i++)
      m[i] = malloc(Y * sizeof(int));

   for(j = 0; j < Y; j++)
      for(i = 0; i < X; i++)
         m[i][j] = i + j;

   return 0;
}
```

9) Se requiere implementar un programa en C que realice la suma horizontal de un vector de floats (ejemplo, dado el vector {10.5, 5.5, 4}, el resultado de la suma horizontal es 20). El vector es muy grande (se requiere memoria dinámica) y el programa debería funcionar para diversos tamaños de vector, simplemente modificando el valor de un literal mediante la directiva #define.

Resolver los siguientes incisos:

a) Implementar una solución utilizando instrucciones escalares.

b) Implementar una solución utilizando instrucciones vectoriales, considerando juegos de instrucciones disponibles que utilicen registros de mayor tamaño, e instrucciones que trabajen con posiciones de memoria alineadas para obtener un mayor rendimiento.

c) Contrastar el tiempo de ejecución de la versión optimizada frente a la versión original.

Notas sobre la compilación del programa vectorial:

Al compilar es necesario indicar el flag correspondiente al juego de instrucciones utilizado (por ejemplo, para avx: -mavx).

10) Dado el siguiente código de programa que utiliza operaciones escalares, escribir su equivalente con funciones intrínsecas vectoriales. Utilizar los juegos de instrucciones disponibles que utilicen registros de mayor tamaño.

```
#include <stdlib.h>
#include <stdio.h>

int main() {
   int i, elementos = 100000000;
   int *v;
   v = malloc(elementos * sizeof(int));

   //no vectorizar este for
   for (i = 0; i < elementos; i++)
```

```
      v[i] = i;

   for (i = 0; i < elementos; i++)
      if (v[i] > 50000000)
         v[i] = v[i] - 5;
      else
         v[i] = v[i] + 5;

   printf("elemento 0: %d\n",v[0]);
   printf("elemento 499999999: %d\n",v[49999999]);
   printf("elemento 500000000: %d\n",v[50000000]);
   printf("elemento 999999999: %d\n",v[99999999]);

   return 0;
}
```

Ejercicio Adicional

Dado el siguiente código de programa que utiliza operaciones escalares, escribir su equivalente con funciones intrínsecas vectoriales. Utilizar los juegos de instrucciones disponibles que utilicen registros de mayor tamaño.

```
#include <stdlib.h>
#include <stdio.h>

int main() {

   int i, elementos = 1000005;
   int *v;

   v = malloc(elementos * sizeof(int));

   // no vectorizar este for
   for (i = 0; i < elementos; i++)
      v[i] = rand() % 10000;

   printf("v[0]=%11d, ", v[0]);
   printf("v[11]=%11d, ", v[11]);
   printf("v[elementos-1]=%11d\n", v[elementos-1]);

   for (i = 0; i < elementos; i++) {
      if(2 * v[i] < i * 10)
         v[i] = v[i] * 2;
      else
         v[i] = (v[i] - 2) * -2;
   }

   printf("v[0]=%11d, ", v[0]);
   printf("v[11]=%11d, ", v[11]);
   printf("v[elementos-1]=%11d\n", v[elementos-1]);

   return 0;
}
```